

# WoS NN on Laplace

Feb. 21<sup>th</sup> 2024

# Background

- for Laplace equation

$$\begin{cases} \Delta u(x) = 0 & x \in \Omega \\ u = g & x \in \partial\Omega \end{cases}$$

- Based on Ito's Lemma (2<sup>nd</sup>-order Taylor expansion), for any Brownian motion  $W_t$  start from specific  $W_0$ , for  $Y_t = u(W_t)$

$$dY_t = \frac{1}{2} \Delta u(W_t) dt + \nabla u(W_t) dW_t$$

- With  $\Delta u(W_t) = 0$ ,

$$Y_{t_{i+1}} = Y_{t_i} + \nabla u(W_{t_i}) \Delta W_{t_{i+1}}, \text{ while } \Delta W_{t_{i+1}} = W_{t_{i+1}} - W_{t_i} \sim N(0, \Delta t)$$

- Use single neural network to approximate  $\nabla u = Z(\cdot, \theta)$

$$Y_\tau = u(W_\tau) = u(W_0) + \sum_{t_i < \tau} Z(W_{t_i}, \theta) \Delta W_{t_{i+1}}$$

where  $W_\tau$  is the exit point on  $\partial\Omega$  and  $\tau$  is the exit time

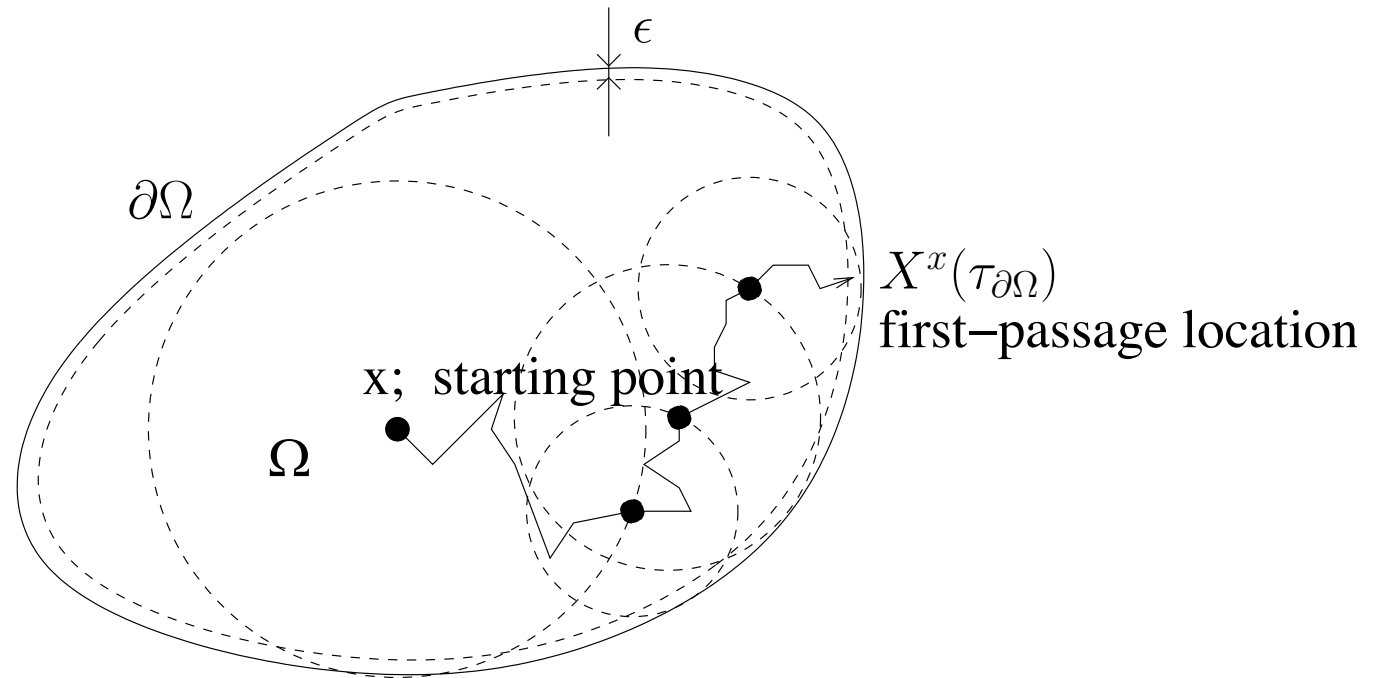
- The loss function is:

$$\mathbb{E}[|Y_\tau - g(W_\tau)|^2]$$

with different Brownian motions from different start places as training inputs.

# Original WoS

- For original WoS,  $u(x) = \mathbb{E}[g(X^x)]$  as in the picture.  
( $u(x_0) = \mathbb{E}[g(W_\tau)]$ )



- Apply NN approximating  $\nabla u$  in each time step. Suppose the current WoS path takes  $k$  steps to reach boundary, it exit each ball at time  $t_i$  by  $x_i$ , and the radius for next ball is  $r_i$ :

$$Y_\tau = u(W_\tau) = u(W_0) + \sum_{i=0}^k Z(x_i, \theta) r_i$$

- This equation works for all single walks. The expectation of changes brought by  $\nabla u$  from inner local to boundary should be 0 when sample from multiple walks

# Original WoS Sample

```
def solver(v, segments, g): #v is the target local/start point, segments are the boundary, g as B.C.
    #...
    # eps for  $\varepsilon$ -shell boundary, nWalks for total walk amount, maxSteps for max steps in one path
    #...
    for i in range(0, nWalks):
        x0 = v
        for step in range(0, maxSteps):
            r = shortestDistance(x0, segments)
            if r < eps:
                break
            theta = random.uniform(0, 2*math.pi)
            x0 = x0 + np.array([r * math.cos(theta), r * math.sin(theta)]) # grab x_i steps within a [[]]
            sumEst += g(x0) # grab g(x0) for loss function right now -> boundary information

    return sumEst/vWalks

# training NN to approximate  $\nabla u$  globally
```

# Settings

- Along the tests, we mainly use  $u = xy$  as the target Laplacian to be approximated.
- On a square region  $[-1, -1] \sim [1, 1]$
- WoS  $\text{eps} = 0.01$
  
- Batch\_size = 1 (manually training one by one now)
- epoch = 1, 5, 30 (for different tests)
- learning\_rate = 0.0001

# 1. Single start place with multiple paths (Start $Y$ is fixed to the WoS result)

- Fixed start place as the center of the region, generated 3000 paths from it, to cover the whole region.
- This local estimation in NN is fixed to be the original WoS estimation result.
- A simple 3-layer fully connect NN to approximate the local gradients:

$$ZNN: x \in R^2 \rightarrow \nabla u(x) \in R^2$$

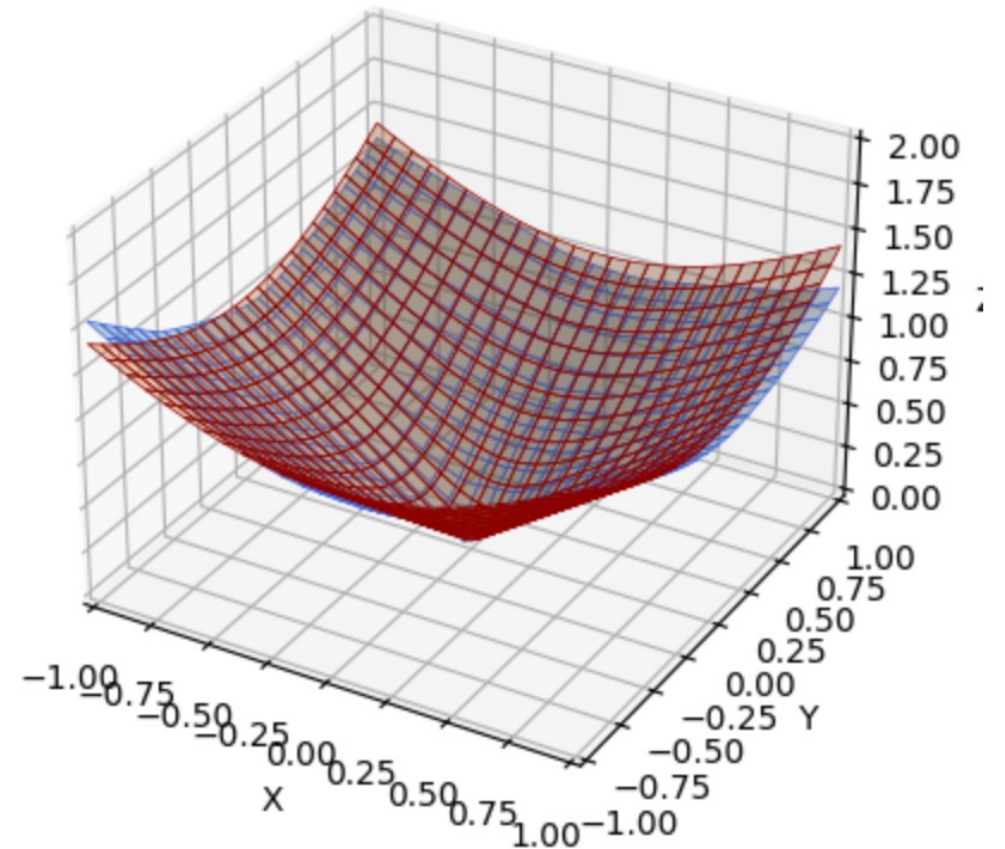
- Forward propagation result in  $(u(W_0))$  as WoS result):

$$Y_\tau = u(W_\tau) = u(W_0) + \sum_{i=0}^k Z(x_i, \theta) \vec{r}_i$$

- Loss function  $E[|Y_\tau - g(W_\tau)|^2]$ , where  $W_\tau$  is the boundary hit obtained from WoS
- Train  $ZNN$  path by path (batch\_size = 1, epoch = 1), obtain a global gradient estimator.

# Some of the test results

- Used around 3500 paths generated from the  $[0, 0]$  center
- The path length is limited by  $\leq 16$  steps
- The step size is limited by  $\leq 0.2$
- $ZNN: x \in R^2 \rightarrow \nabla u(x) \in R^2$   
as a 3-layer dense NN ([32, 64, 128])
- Plot the result gradients' norm on  $200 \times 200$  point on region  $[-1, 1] \times [-1, 1]$
- red one is the correct gradient norms, blue one is the approximated gradient norms
- 5mins – 1 epoch



# Next Step

- Initial idea: use the WoS results to improve the NN estimation.
  - Use WoS estimated  $u(x_0)$  as an input of the network rather than a parameter to be optimized
  - Requires multiple path generated from each local point to cover the whole region, and fewer start points -> may cannot cover the region and unaccurate.
  - adaptive sampling?
- A large amount of random start points with only 1 path generated for each. WoS is not actually working, but just as a path sampler.
  - NN is not based on local estimations given by WoS, but willing to generate a global gradient estimator and a global local estimator -> only need paths to cover the region.
  - convergence? Math proving?

## 2. Multiple start places with 1 path for each (Start $Y$ is approximated by YNN)

- Sample paths start from  $100 * 100$  uniformly distributed points on  $[-1, 1]^2$ .
- For each local, walk only one path on it. Each path satisfies the following:

$$Y_\tau = u(W_\tau) = Y(x_0; \theta) + \sum_{i=0}^k Z(x_i; \theta) \vec{r}_i$$

- Create a YNN similar to ZNN to approximate the local results, along with ZNN to approximate gradients:

$$\begin{aligned} ZNN: x \in R^2 &\rightarrow \nabla u(x) \in R^2 \\ YNN: x \in R^2 &\rightarrow u(x) \in R^1 \end{aligned}$$

- Loss function  $E[|Y_\tau - g(W_\tau)|^2]$ , where  $W_\tau$  is the boundary hit obtained from WoS
- Train *YNN and ZNN* path by path (batch\_size = 1, epoch = 5), obtain a global gradient estimator.

# Network to approximate boundaries for paths

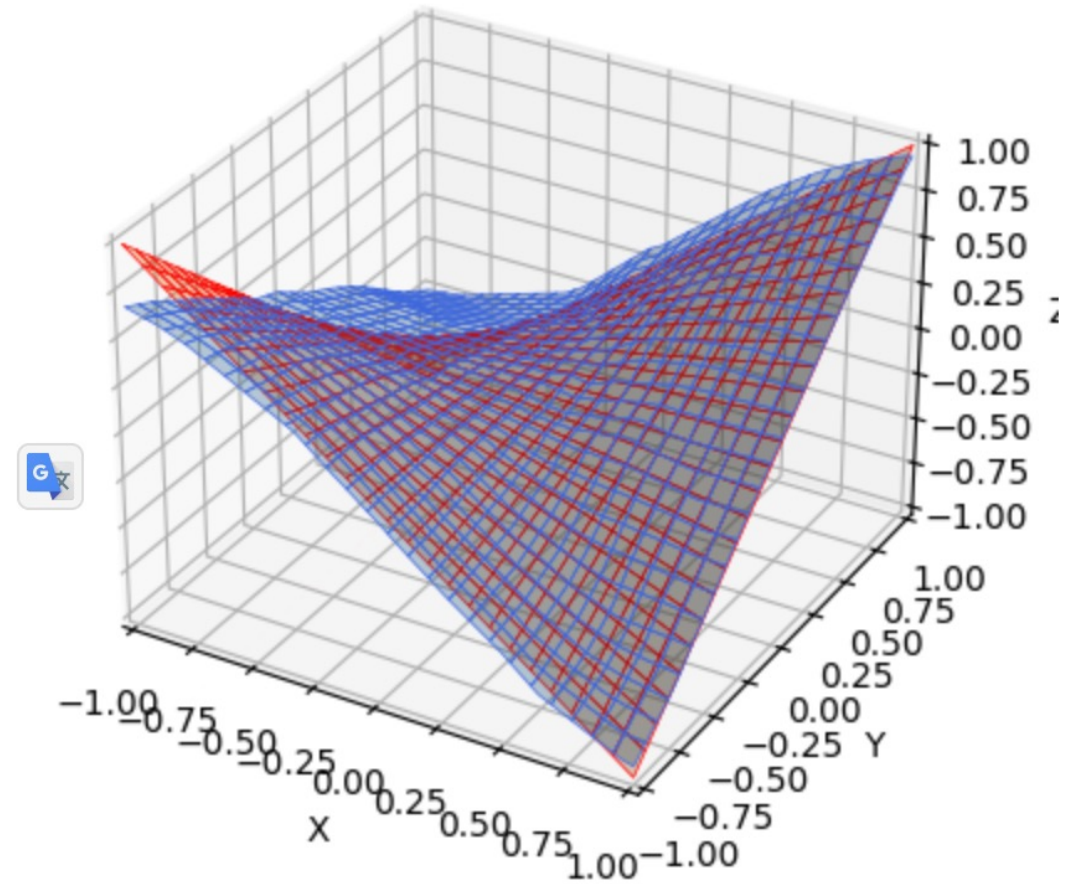
```
class Ysolver(keras.Model):
```

```
    def __init__(self, layerZ, layerY):  
        super(Ysolver, self).__init__()  
        self.Ymodel = YNN(layerY)  
        self.Zmodel = ZNN(layerZ)
```

```
    def call(self, path, training):  
        y_pred = self.Ymodel(np.array(path[0]).reshape(1,2).astype("float32"), training)  
        for i in range(len(path) - 1): ###input path should be (-1, 2) shaped nparray, no len()  
            z = self.Zmodel(np.array(path[i]).reshape(1,2).astype("float32"), training) #obtain gradients for steps  
            y_pred += tf.reduce_sum(z * (path[i + 1] - path[i]), 1, keepdims=True) #accumulating  
        return y_pred
```

# Some of the test results

- Used around 8000 paths generated from  $100 \times 100$  points, which uniformly distributed on the region
- The path length is limited by  $\leq 16$  steps
- $ZNN: x \in R^2 \rightarrow \nabla u(x) \in R^2$   
as a 3-layer dense NN ([32, 12, 12])
- $YNN: x \in R^2 \rightarrow u(x) \in R$   
as a 3-layer dense NN ([12, 12, 12])
- Plot the result local estimation (YNN) on  $200 \times 200$  point on region  $[-1, 1] \times [-1, 1]$
- red one is the correct locals, blue one is the approximated local results.



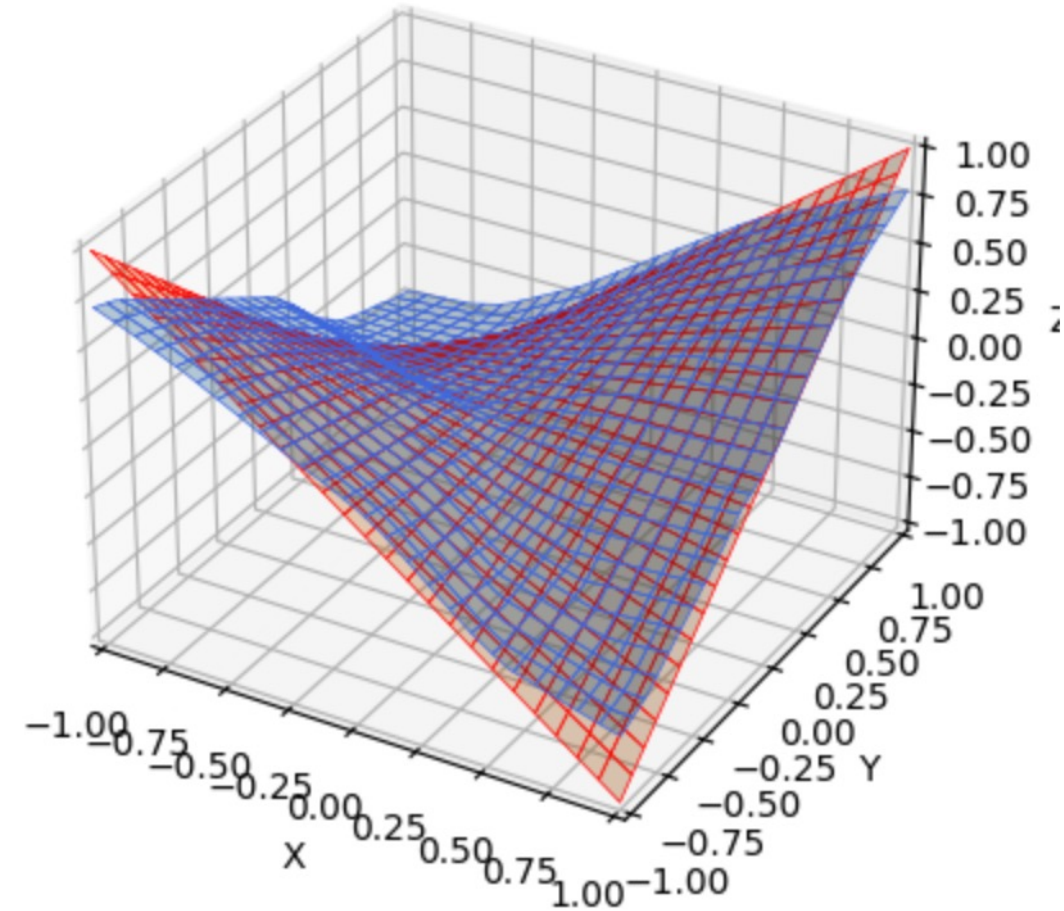
# Variations

- A problem is the training time is too long, for 8000 paths trained one by one. (8mins per epoch)

## 1. Use fewer paths and train with more epochs:

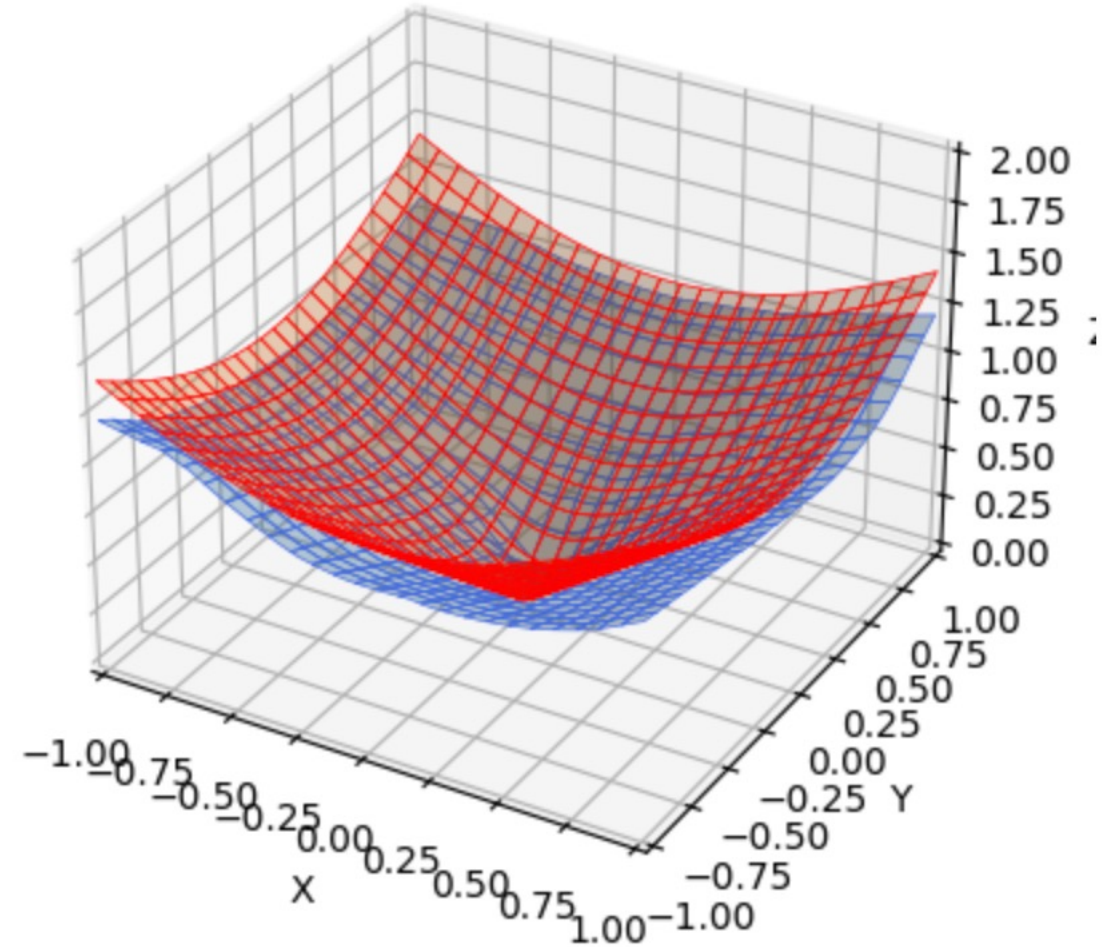
- On range  $[-1, 1] \times [-1, 1]$ , sampled evenly with step size 0.1, then 1 path for each local sample -> around 300 paths
- epoch = 30, YNN = [12, 12, 12], ZNN = [32, 12, 12], training time < 15mins
- red one is the correct locals, blue one is the approximated local results.

## 2. Vectorization



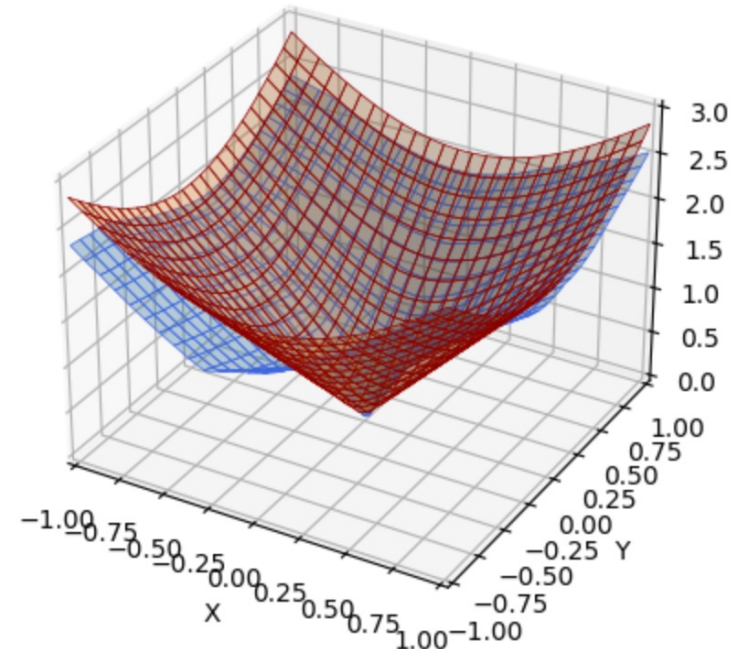
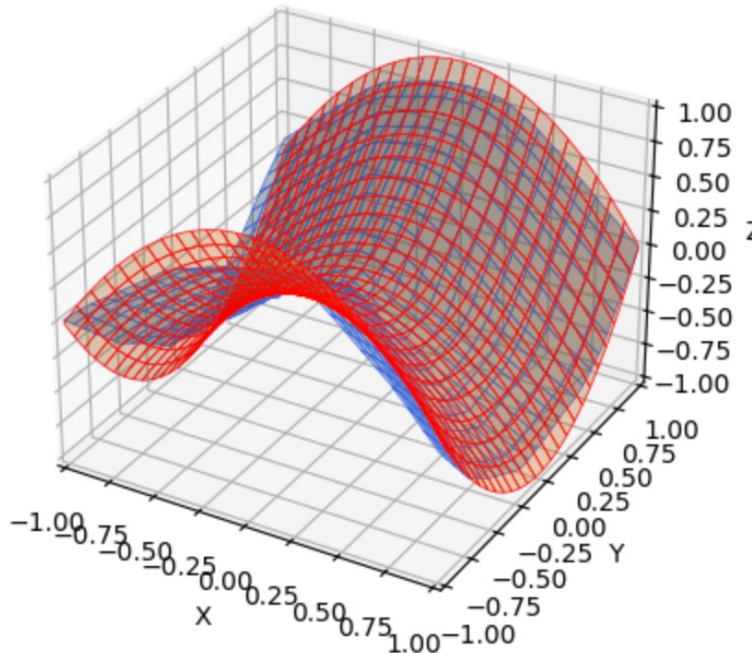
# Gradient results

- Here is the gradient results approximated by ZNN in the previous case.
- red one is the correct gradient norms, blue one is the approximated gradient norms
- The approximated gradient norm is still smaller than the real gradients -> not trained enough yet, may need more epoch



# Attempt with other B.C.

- Change the B.C. (which is also the target to be approximated) to  $u = y^2 - x^2$ , which gives  $\Delta u = 2 - 2 = 0$
- sampled  $40 \times 40$  local start points evenly distributed among  $[-1, 1] \times [-1, 1]$ , 1300 paths successfully generated. epoch = 30
- Left picture is for solution  $u$ , right picture is for gradient  $\Delta u$ .



# What affect the result

- In the first case, path lengths and step lengths affects the results
  - large path length means rugged path, large step lengths bring in errors when accumulating gradients
  - This requires the input quality -> to achieve enough input data? 20000 -> 4000
  - In the second case, step lengths is no longer sensitive. Why?
- Hyper parameters
  - activation functions, learning rate, NN structure
  - In the second case, more epoch & more neurons needed in the training
- More tests with different B.C.
- Batch normalization makes things worse, why?
- Vectorization to greatly accelerate the training.

### 3. Another attempt of using the result of WoS to train a Y approximator directly

- Use the result of WoS to train a Y approximator directly, without training a global gradient estimator.
- Sampled  $100 \times 100$  locals uniformly distributed among the region  $[-1, 1] \times [-1, 1]$ . For each local, generated 100 paths to estimate the local result  $u(x)$

- Use local points and the WoS results to train a YNN:

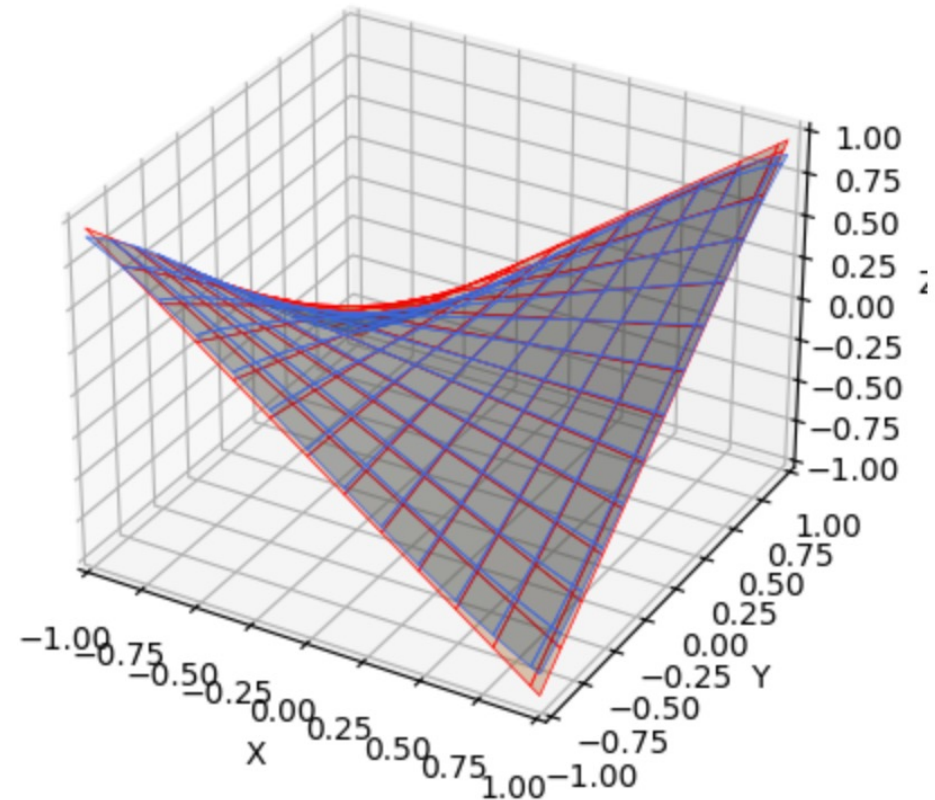
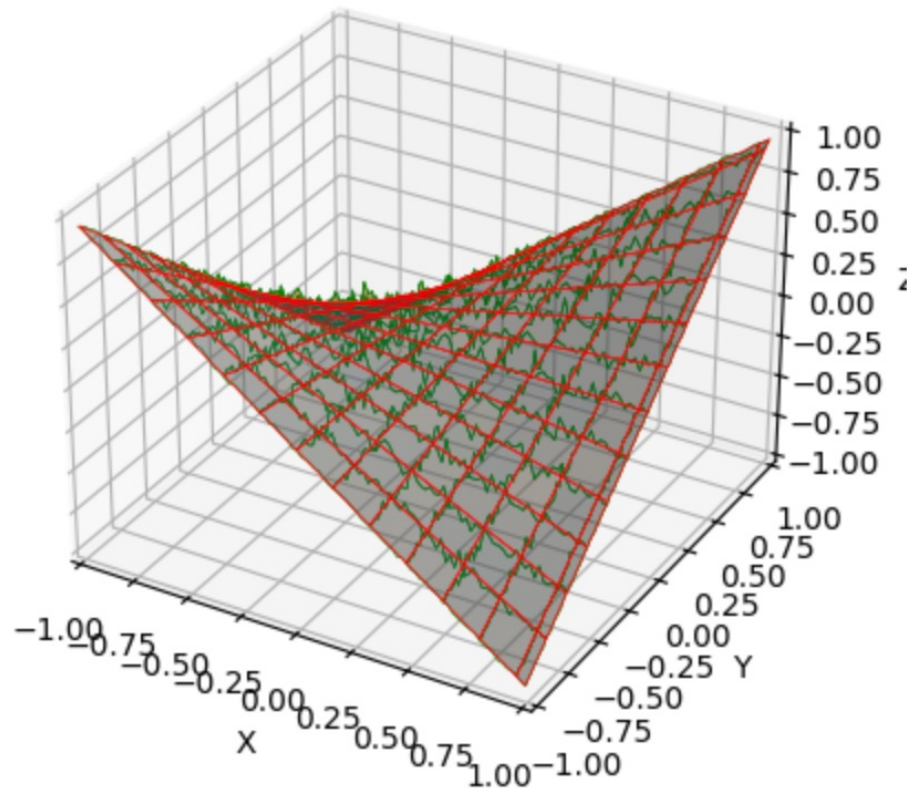
$$YNN: x \in R^2 \rightarrow u(x) \in R$$

as a 3-layer dense NN ([256, 128, 64])

- batch\_size = 64, epochs = 5
- A time problem is the WoS!  $100 \times 100 \times 100$  samples of paths now, limited by curse of dimensionality for harder cases/higher dimensions?

# Some of the test results

- Left is the WoS result V.S. target result, right is the YNN result V.S. target result
- NN training smoothed the WoS results.



# Next Step to do

- Finished:
  - 1 center point with multiple paths, use WoS local estimation as an input to the network and train the NN of the global gradient
  - Multiple start points with 1 path for each, local solutions act as parameter to be optimized
- Wrap up:
  - More test with different hyper parameters
  - Vectorization -> batch utilize
  - time complexity analyze, case 2 comparing to case 3
- Source term situation
  - Poisson equation -> Poisson Boltzman equation with absorption
- inner-balls inside large spheres to cut the Brownian motion
  - Now cannot sample from inner-balls uniformly. Need specific distribution based on the more inner samples.
  - Prof. Arash code
- Ito process instead of simple Brownian

# Source term contribution

- Other 2<sup>nd</sup>-order elliptic equations with Laplace (Poisson here)

$$\begin{cases} \Delta u(x) = f & x \in \Omega \\ u = g & x \in \partial\Omega \end{cases}$$

The source term was associated with time in Feynman-Kac formula:

$$u(x_0) = E[g(W_\tau)] - E\left[\int_0^\tau f(W_t)dt\right]$$
$$Y_{t_{i+1}} = Y_{t_i} + \frac{1}{2} \int_{t_i}^{t_{i+1}} f(W_t)dt + \nabla u(W_{t_i})\Delta W_{t_{i+1}}$$

which include the gap of time -> requires small time steps

- WoS for Poisson equation enables us to estimate the source term contribution by in-ball samples (**sampled by Green's distribution**), without accumulate it along the process time.

$$Y_{t_{i+1}} = Y_{t_i} + \frac{1}{2} f(S_i)(t_{i+1} - t_i) + \nabla u(W_{t_i})\Delta W_{t_{i+1}}$$

where  $t_i$  is the exit time of i-th ball and  $S_i$  is the in-ball sample of i-th sphere:

$$Y_\tau = u(x_0) + \sum_{i=0}^k \frac{1}{2} f(S_i)(t_{i+1} - t_i) + \sum_{i=0}^k Z(x_i, \theta)r_i$$

# Source term contribution

- Get rid of time fully? Go through Elepov paper again to see how they utilize in-ball samples.
  - Is this really good?  $Y_{t_{i+1}} = Y_{t_i} + \frac{1}{2} f(S_i)(t_{i+1} - t_i) + \nabla u(W_{t_i}) \Delta W_{t_{i+1}}$
- In-ball samples based on Green's function
  - distribution & inversion
- Time samplings if really needed:
  - HITTING TIME FOR BESSEL PROCESSES—WALK ON MOVING SPHERES ALGORITHM (WOMS)
- Combining with inner-balls inside large spheres to cut the Brownian motion

| PDE                                                                                                           | Feynman-Kac                                                                                                                                                                      | WoS estimation | Stochastic ver. | Stochastic Approximation | NN approximation |
|---------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|-----------------|--------------------------|------------------|
| $\begin{cases} \Delta u = 0 \\ u = g \end{cases}$                                                             | $\mathbb{E}[g(W_\tau)]$                                                                                                                                                          |                |                 |                          |                  |
| $\begin{cases} \Delta u = f \\ u = g \end{cases}$                                                             | $\begin{aligned} &\mathbb{E}[g(W_\tau)] \\ &- \mathbb{E}\left[\int_0^\tau f(W_t)dt\right] \end{aligned}$                                                                         |                |                 |                          |                  |
| $\begin{cases} \Delta u - \sigma u = f \\ u = g \end{cases}$                                                  | $\begin{aligned} &\mathbb{E}[e^{-\sigma\tau}g(W_\tau)] \\ &- \mathbb{E}\left[\int_0^\tau e^{-\sigma t}f(W_t)dt\right] \end{aligned}$                                             |                |                 |                          |                  |
| $\begin{cases} \nabla(\alpha(x)\nabla u(x)) \\ + \vec{w}\nabla u(x) + \\ \sigma(x)u = f \\ u = g \end{cases}$ | $\begin{aligned} &\mathbb{E}\left[e^{-\int_0^\tau -\sigma(X_t)dt}g(X_\tau)\right] \\ &- \mathbb{E}\left[\int_0^\tau e^{\int_0^t -\sigma(X_z)dz} * f(X_t)dt\right] \end{aligned}$ |                |                 |                          |                  |